# Inference Simulations

**PSYC 201A, Simulation Lab 2**

Lumi Kang

2026-10-20

## Table of contents


Tips before getting started . . . 2

**Setup** **2**
Import the libraries . . . 2

**Part 1: Simulating experimental data** **3**
Define the simulation function . . . 3
Make data frames where we have small or larger samples of tea data for ONE experiment 3
In this first draw, was it significant with N=9 or N=24 per group? . . . 4
Simulate 1000 experiments . . . 5
Summarize both of these simulations . . . 5
Plot difference for low-n . . . 6
Plot difference for higher-n . . . 7

**Part 2: Simulating the null distribution** **9**
Method 1: Simulation . . . 10
Method 2: Shuffling . . . 11
Let's shuffle the labels of ONE experiment, 1000 times . . . 11
Visualizing what happens when we shuffle . . . 12

**Part 3: Calculating confidence intervals** **16**
Confidence intervals . . . 16
Get the data . . . 16
Calculate a CI on the effect (difference between conditions) . . . 17
Plotting CIs by hand for each condition . . . 18

**Part 4: Simulating p-values across multiple experiments** **20**
When there is no actual effect . . . 20

Now for an experiment when there is actually an effect . . . . . 27

**Part 5: Power curves** **28**

**Exercises** **31**

### Tips before getting started

This is a document made to accompany some simulations for the Inference workshop in Psych 201a. The goal of this document is to continue learning in R/tidyverse but also to gain hands on experience simulating and manipulating data.

Open this file by double-clicking `inference_simulations_lab_2.Rproj` first, then opening it from the Files pane. The only package it needs is the tidyverse, which you installed in getting-started-with-r.

To understand what a function does, type `?function_name` in the Console (e.g., `?rnorm`).

`error: true` is set at the top of this file, so it still renders while some chunks are unfinished: those chunks show their error in red. Several later chunks depend on objects you create earlier (`tea_data_highn`, `samps_highn`, `null_model`), so work from top to bottom.

## Setup

```
set.seed(123) # good practice to set a random seed, or else different runs get you different
```

### Import the libraries

```
install.packages("tidyverse")
```

```
library(tidyverse)
```

## Part 1: Simulating experimental data

### Define the simulation function

This makes "tea data", a tibble (dataframe) where there are a certain number of people in each condition (default = 48, i.e., n_total, with n_total/2 in each half)

The averages of the two conditions are separated by a known effect ("delta") with some variance ("sigma"). You can change these around since we're simulating data!

So, here, DELTA is the difference is the tea ratings, on average, between the two conditions, in likert scale points. For simplicity, the scale is from 1-10.

```
make_tea_data <- function(n_total = 48,
                          sigma = 1.25, # sd of the distribution
                          delta = 1.5) {  # difference between distributions
  n_half <- n_total / 2 # half of the total subjects  = #/n subs per condition
  tibble(condition = c(rep("milk first", n_half), rep("tea first", n_half)),
         rating = c(round(
           rnorm(n_half, mean = 3.5 + delta, sd = sigma)
         ),
         round(rnorm(
           n_half, mean = 3.5, sd = sigma
         )))) |>
    mutate(rating = if_else(rating > 10, 10, rating), # random sampling could mean the scale
           # truncate if greater than max/min of rating scale
           rating = if_else(rating < 1, 1, rating))
}
```

### Make data frames where we have small or larger samples of tea data for ONE experiment

```
# here, we're calling our custom function, and specifying different inputs than the defaults
tea_data <- make_tea_data(n_total = 18, delta = .5)
```

Now, make a new data frame (call it `tea_data_highn`) with an N of 48, or something higher if you want. You need to call the custom function. Keep the same small effect (`delta = .5`): we come back to this experiment in Parts 2 and 3.

```
tea_data_highn <- make_tea_data(n_total = 48, delta = .5)
```

To do: OK, look at these data. How long are they, what are the column names? Look at them in your console by copy and pasting the variable names into R.

```
# There are 24 milk-first and 24 tea-first. The variable names are "condition" and "rating."
```

Call `summary(tea_data)` and look at the outputs. Call `head(tea_data)` to look at the first few rows.

To do: Write basic tidyverse code to calculate the mean of each condition (hint: use `group_by` and `summarize`).

```
tea_first_df <- tea_data |>
  group_by(condition) |>
  summarize(
    mean = mean(rating),
    sd = sd(rating),
    n = n()
  )

tea_first_df2 <- tea_data_highn |>
  group_by(condition) |>
  summarize(
    mean = mean(rating),
    sd = sd(rating),
    n = n()
  )
```

### In this first draw, was it significant with N=9 or N=24 per group?

OK, we can do t-tests already! I've done these within a pipe (I often use the old pipe `%>%`, which does the same thing as `|>`; don't mind the difference).

To do: Run these and look at the outputs by typing `out_low_n` and `out_high_n` in the console.

```
out_low_n <- tea_data %>%
  t.test(rating ~ condition, data = ., var.equal = TRUE)

out_high_n <- tea_data_highn %>%
  t.test(rating ~ condition, data = ., var.equal = TRUE)
```

R practice: Use the `$` operator and then press `tab` to see how you can get specific values (i.e., the `t` or `p` value) from the outputs.

Try writing the outputs of the t-test in a reproducible report style paragraph.

```
out_low_n <- tea_data %>%
  t.test(rating ~ condition, data = .,  var.equal = TRUE)

out_high_n <- tea_data_highn %>%
  t.test(rating ~ condition, data = ., var.equal = TRUE)
```

### Simulate 1000 experiments

Now let's simulate 1000 experiments...where you have 18 participants per experiment with an average difference of 1.5 points in tea deliciousness on average

Below is kind of annoying and complex tidyverse, so I'm doing it for you. I'd have to look this up how to do it.

```
samps <- tibble(sim = 1:1000) |>
  mutate(data = map(sim, \(i) make_tea_data(n_total = 18, delta = 1.5))) |>  # simulate
  unnest(cols = data) # wrangle
```

OK, copy this but now change it so that you 48 participants per experiment

```
samps_highn <-tibble(sim = 1:1000) |>
  mutate(data = map(sim, \(i) make_tea_data(n_total = 48, delta = 1.5))) |>
  unnest(cols = data)
```

### Summarize both of these simulations

Here, we take these simulated experiments, group by `sim` and `condition`, and summarize the average ratings per experiment.

Then, for each experiment, calculate the mean difference between the milk first vs. tea first conditions.

To do: Comment what each line of code is doing to check your understanding, and talk to a partner about it.

Bonus: Figure out a way to do this that is better than I what I did (some mix of base R and tidyverse, I'm sure there's a tidyverse only way)

```r
tea_data_summary <- samps |>
  group_by(sim, condition) |>
  summarise(mean_rating = mean(rating)) |>
  group_by(sim) |>
  summarise(delta = mean_rating[condition == "milk first"] -
              mean_rating[condition == "tea first"])
```

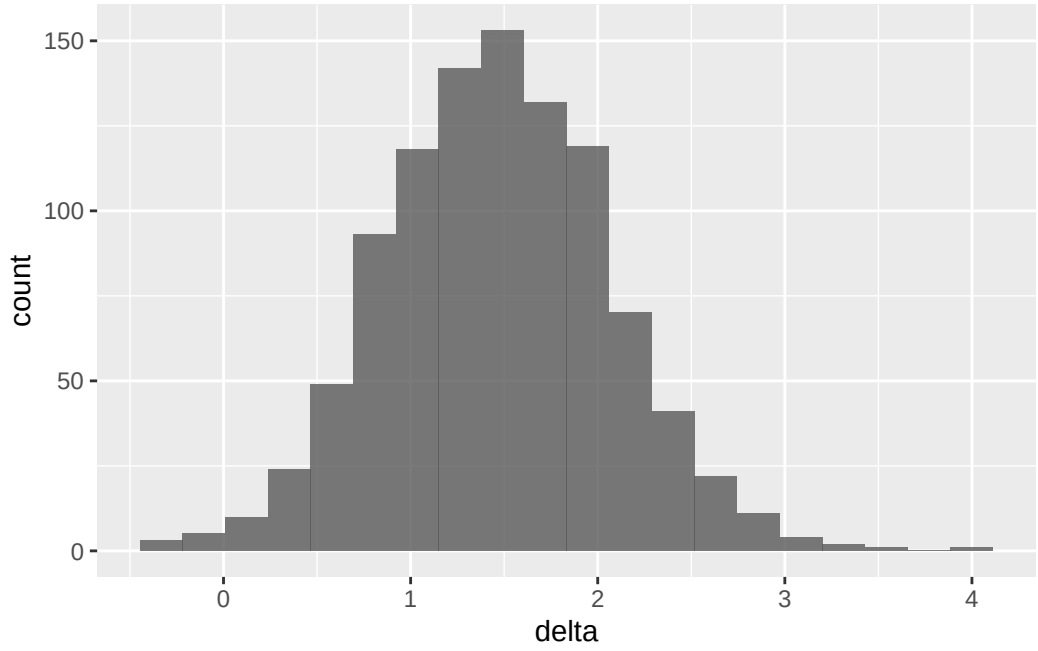
```r
tea_data_highn_summary <- samps_highn |>
  group_by(sim, condition) |>
  summarise(mean_rating = mean(rating)) |>
  group_by(sim) |>
  summarise(delta = mean_rating[condition == "milk first"] -
              mean_rating[condition == "tea first"])
```

### Plot difference for low-n

Let's make a plot to plot the differences in ratings across conditions for the low-n condition. You'll need to use ggplot or `hist`

```r
low_n_plot <- ggplot(data = tea_data_summary, aes(x = delta)) +
  geom_histogram(alpha = .8, bins = 20)

print(low_n_plot)
```

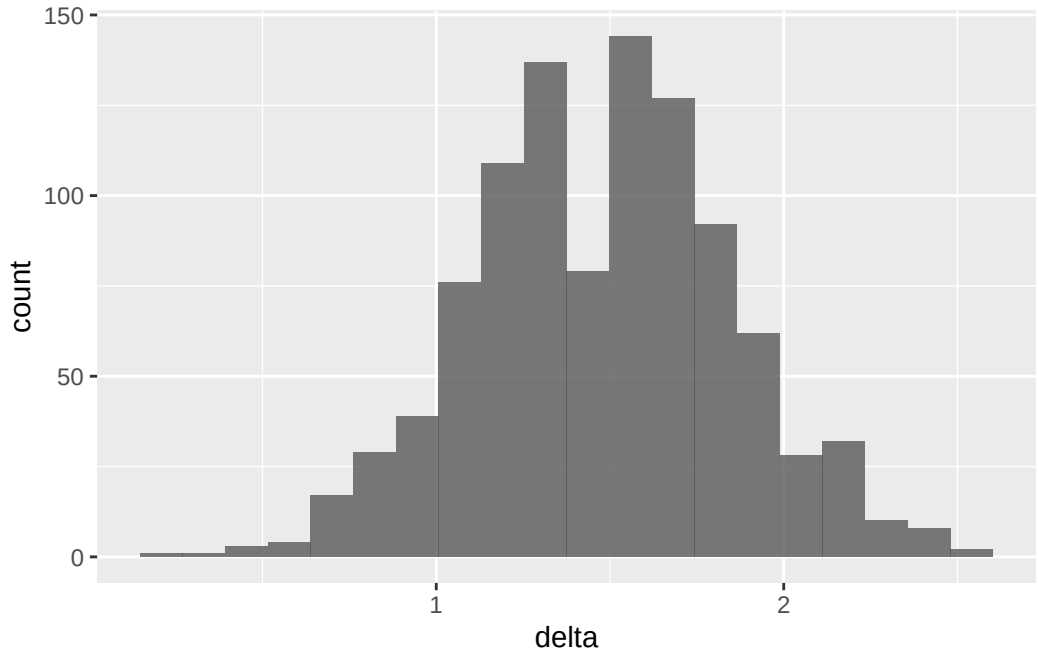



**Plot difference for higher-n**

To do: What's different about this distribution vs the one we just plotted?

```
high_n_plot <- ggplot(data = tea_data_highn_summary, aes(x = delta)) +
  geom_histogram(alpha = .8, bins = 20)

print(high_n_plot)
```




```
# The small sample size simulation somewhat looks more like a normal distribution, but the la
```

**Bonus** What happens if you run it again, but this time try changing the variance / mean of the effect when you change the `delta` and `sigma` values in the data simulation.

```
tea_data_new <- make_tea_data(n_total = 18, delta = 1, sigma = 1)

new_df <- tea_data_new |>
  group_by(condition) |>
  summarize(
    mean = mean(rating),
    sd = sd(rating),
    n = n()
  )

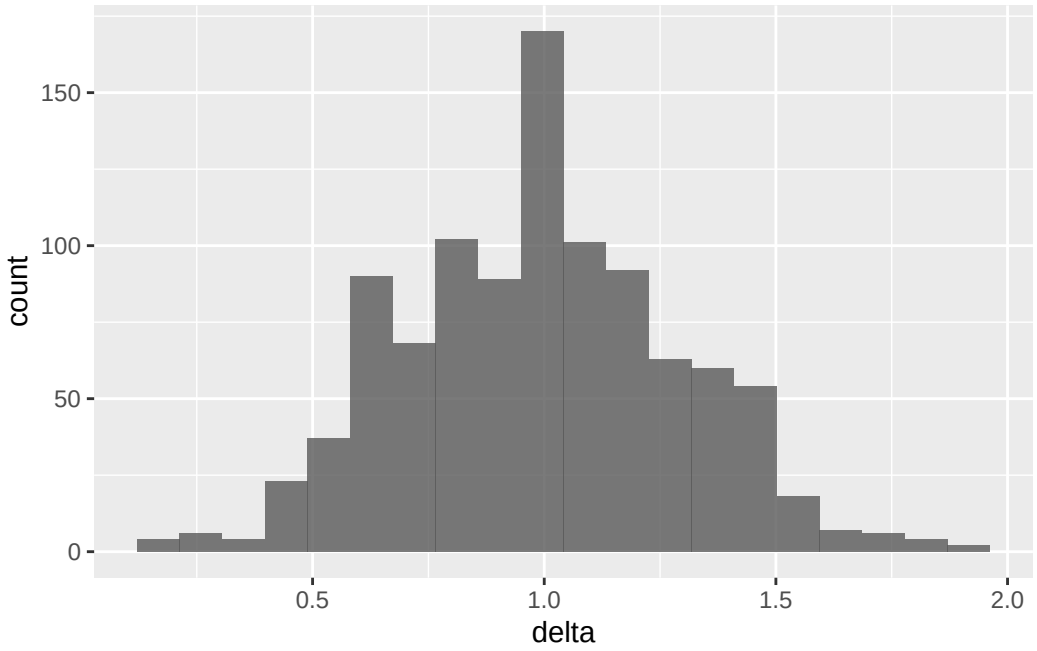
new_ttest <- tea_data_new %>%
  t.test(rating ~ condition, data = ., var.equal = TRUE)

samps_new <-tibble(sim = 1:1000) |>
  mutate(data = map(sim, \(i) make_tea_data(n_total = 48, delta = 1, sigma = 1))) |>
  unnest(cols = data)
```

```r
new_summary <- samps_new |>
  group_by(sim, condition) |>
  summarise(mean_rating = mean(rating)) |>
  group_by(sim) |>
  summarise(delta = mean_rating[condition == "milk first"] -
              mean_rating[condition == "tea first"])

new_plot <- ggplot(data = new_summary, aes(x = delta)) +
  geom_histogram(alpha = .8, bins = 20)

print(new_plot)
```




What happens if you vary the random seed? How much does this change what you find?

## Part 2: Simulating the null distribution

Visualizing the null distribution. How can we actually visualize the NULL hypothesis and distribution? It's a little tricky, but we can rely on simulation to help us.

### Method 1: Simulation

First, do the same thing as you did before - simulating 1000 experiments – but NOW simulate no differences between conditions. Remember, it's the null model because DELTA (i.e., differences in conditions) is ZERO. Keep these dataframes - we will use them later for estimating p-values

Call the simulated experiments `null_model` and use 18 participants per experiment (`n_total = 18`); Part 4 uses `null_model`. Then summarize it the same way you summarized `samps` above.

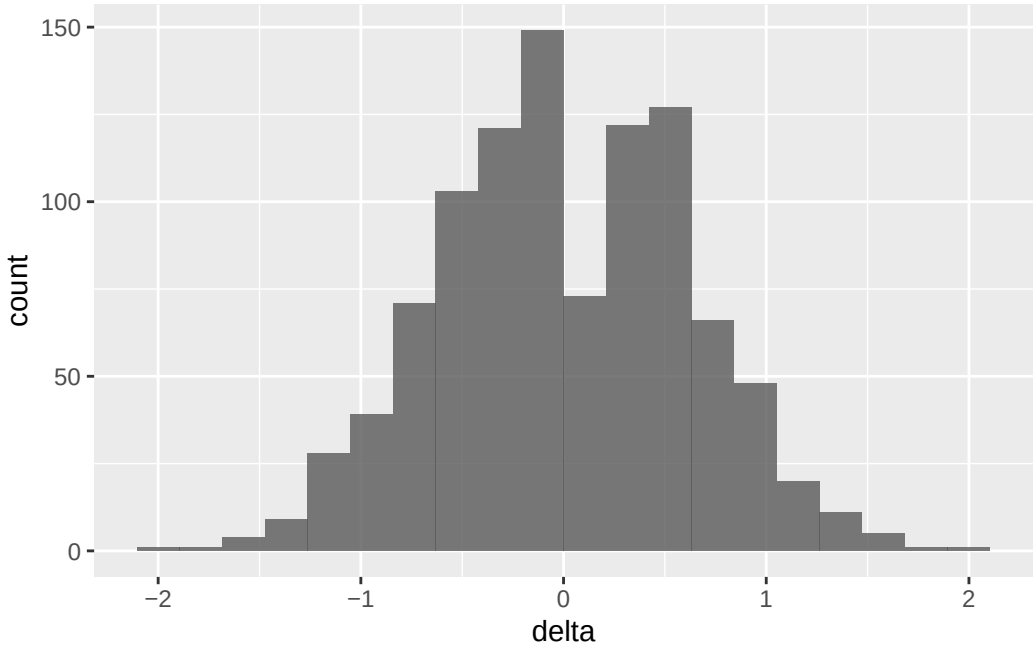
```
# Simulate the null distribution
null <- tibble(sim = 1:1000) |>
  mutate(data = map(sim, \(i) make_tea_data(n_total = 18, delta = 0))) |>
  unnest(cols = data)

null_summary <- null |>
  group_by(sim, condition) |>
  summarize(mean_rating = mean(rating)) |>
  group_by(sim) |>
  summarise(delta = mean_rating[condition == "milk first"] - mean_rating[condition == "tea fi
```

Visualize what this null distribution actually looks like.

```
null_plot <- ggplot(data = null_summary, aes(x = delta)) +
  geom_histogram(alpha = .8, bins = 20)

print(null_plot)
```




### Method 2: Shuffling

We can also calculate the distribution of the difference between conditions when we've shuffled the condition labels. This is referred to as the *empirical null hypothesis* (our H0, since we're comparing two conditions with a two-sample t-test)

#### Let's shuffle the labels of ONE experiment, 1000 times

```
tea_data_highn_shuffled <- tibble(sim = 1:1000) |> # 1000 shuffles
  mutate(data = map(sim, \(i) tea_data_highn)) |> # the SAME experiment each time
  unnest(cols = data) |>
  group_by(sim) |> # within each shuffle...
  mutate(condition_shuffled = sample(condition)) # ...randomly reassign the condition labels
```

We shuffle `tea_data_highn`, the one experiment whose $p$-value we want below, not 1000 different experiments. The question a permutation test asks is: "if the labels in *this* dataset meant nothing, how big a difference would we see just by chance?"

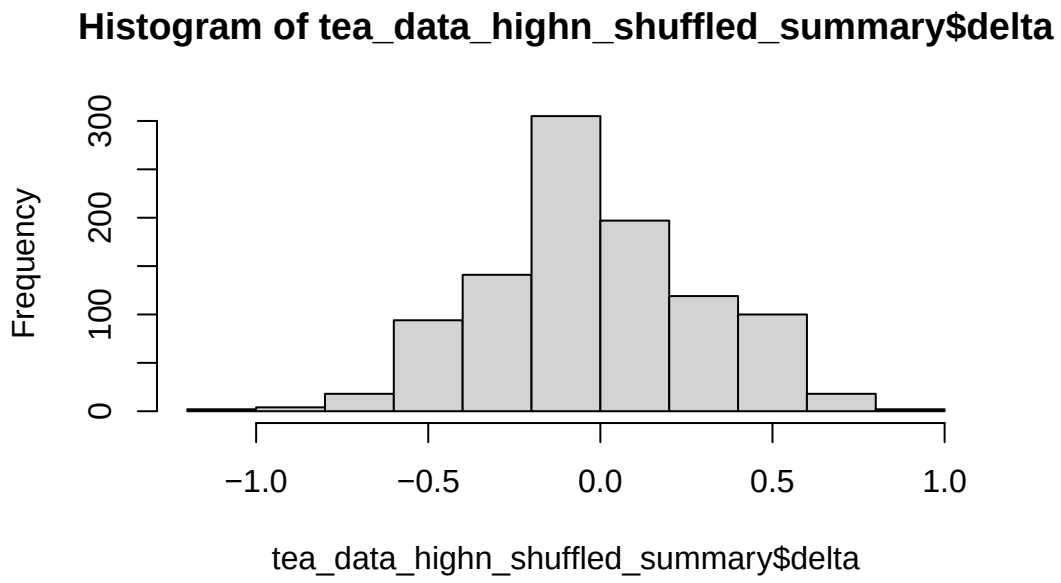
To do: check your understanding – what is "sim" here? How is it different from `sim` in `samps_highn`?

```r
tea_data_highn_shuffled_summary <- tea_data_highn_shuffled %>%
  group_by(condition_shuffled, sim) %>%
  summarize(mean_rating = mean(rating),
            sd = sd(rating))

# overwrites this variable
tea_data_highn_shuffled_summary <- tea_data_highn_shuffled_summary %>%
  group_by(sim) |>
  summarise(delta = mean_rating[condition_shuffled == "milk first"] -
              mean_rating[condition_shuffled == "tea first"])
```

To do: what does this histogram look like and why?

```r
hist(tea_data_highn_shuffled_summary$delta) #what does this histogram look like?
```



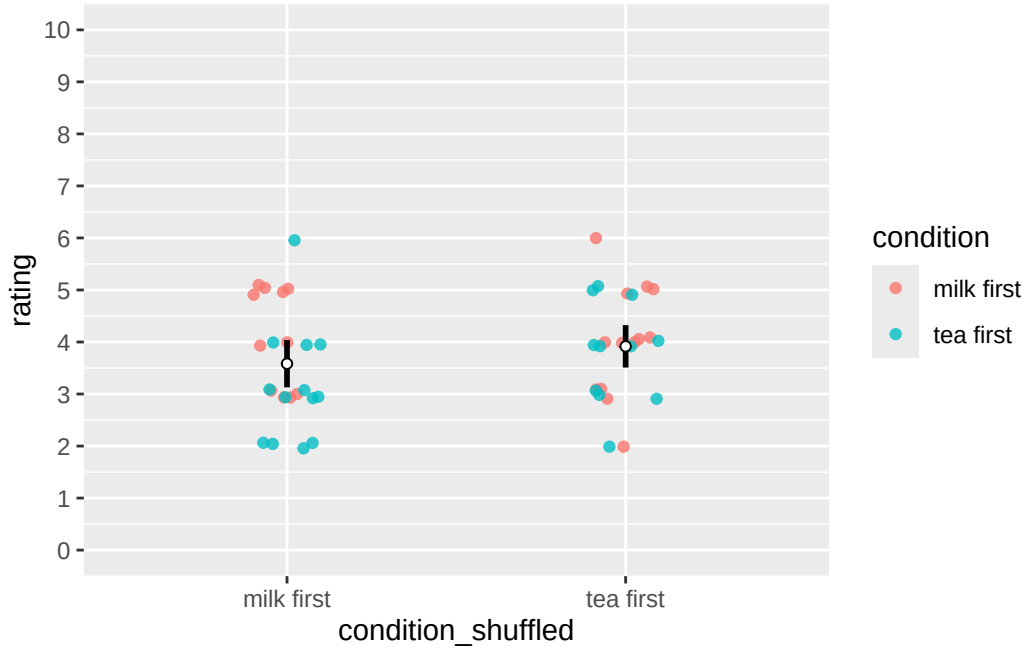

```r
# Looks like a normal distribution
```

#### Visualizing what happens when we shuffle

OK, now see what happens to our raw data – this is just one of the shuffles. The color refers to the ORIGINAL label before we shuffled, but our condition difference is gone

The color refers to the ORIGINAL condition The x-axis refers to the SHUFFLED condition

```
ggplot(data = tea_data_highn_shuffled %>% filter(sim==3),  # can change the shuffle number he
       mapping = aes(x = condition_shuffled, y = rating))+
  geom_point(mapping = aes(color = condition), # color
             alpha=.8,
             position = position_jitter(height = .1,
                                        width = 0.1)) +
  stat_summary(fun.data = mean_se, fun.args = list(mult = 1.96), # mean +/- 1.96 SE: an appro
               geom = "linerange",
               linewidth = 1) +
  stat_summary(fun = "mean", # this calculates the average
               geom = "point",
               shape = 21,
               color = "black",
               fill = "white") +
  scale_y_continuous(breaks = 0:10,
                     labels = 0:10,
                     limits = c(0, 10))
```




The idea is now that we can get a sampling distribution of the difference in the means between the two conditions (assuming that the null hypothesis were true), by randomly shuffling the labels and calculating the difference in means (and doing this many times).

What we get is a distribution of the differences we would expect, if there was no effect of condition.

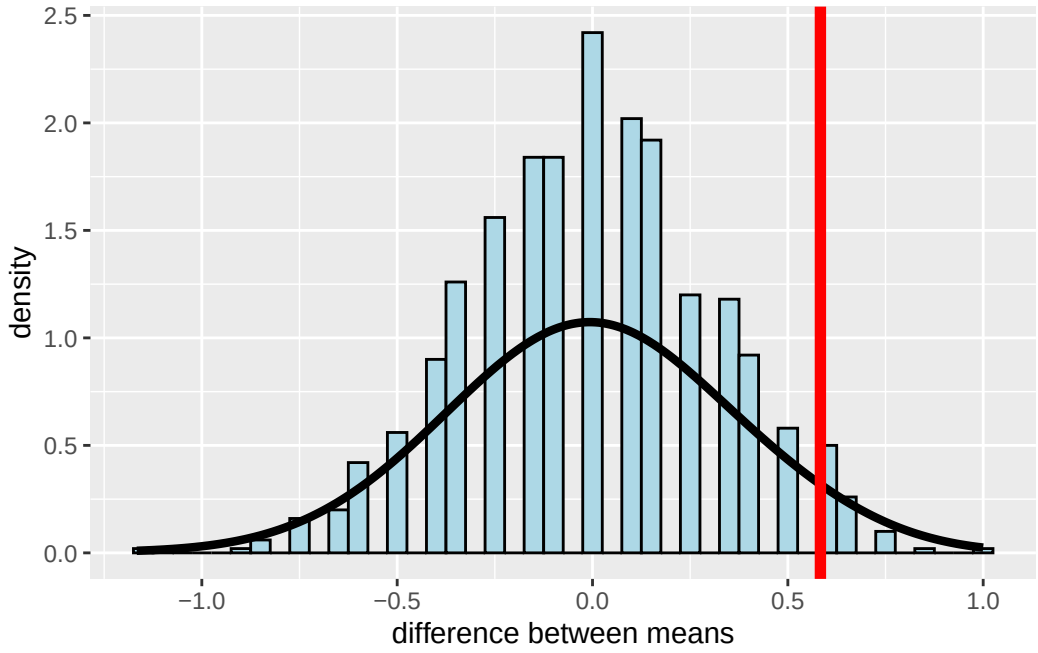
First, let's calculate the actual difference in the experiment we shuffled (we simulated it with a true effect of delta = 0.5)

```
difference_actual <- tea_data_highn |>  # in ONE experiment
  group_by(condition) |>
  summarize(mean_rating = mean(rating)) |>
  summarise(delta = mean_rating[condition == "milk first"] -
              mean_rating[condition == "tea first"]) |>
  pull(delta)
```

Plot the distribution of the differences across all of our shuffled iterations

And in red – is the numerical value from when there was an effect (because we simulated it that way)

```
ggplot(data = tea_data_highn_shuffled_summary, aes(x=delta)) +
  geom_histogram(aes(y = after_stat(density)),
                 color = "black",
                 fill = "lightblue",
                 binwidth = 0.05) +
  stat_density(geom = "line",
               linewidth = 1.5,
               bw = 0.2) +
  geom_vline(xintercept = difference_actual, color = "red", linewidth = 2) +
  labs(x = "difference between means")
```



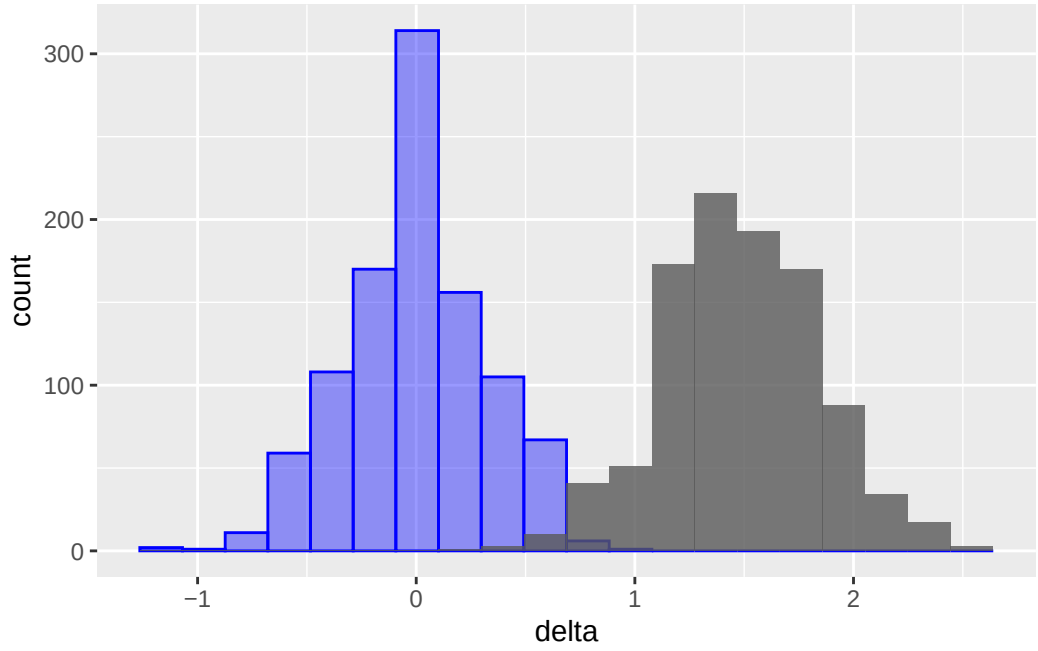

And we can THEN calculate the p-value by using some basic data wrangling – it is defined as the proportion of condition differences (treatment - control) that were as or more extreme than the one we observed.

```
# two-sided: shuffled differences at least as far from zero as the observed one
tea_data_highn_shuffled_summary |>
  summarize(p_value = mean(abs(delta) >= abs(difference_actual)))
```

```
# A tibble: 1 x 1
  p_value
    <dbl>
1    0.09
```

You can also see this if you plot the distributions of the null vs empirical simulations next to each other. Blue = the shuffled (null) differences, which are appropriately centered at zero. Gray = the 1000 simulated experiments from `samps_highn`, where the true effect is delta = 1.5. This should remind you of the cohen's d web app we looked at!

```
ggplot(data=tea_data_highn_shuffled_summary, aes(x=delta)) +
  geom_histogram(alpha=.4, bins=20, color='blue', fill='blue') +
  geom_histogram(alpha=.8, bins=20, data=tea_data_highn_summary)
```




## Part 3: Calculating confidence intervals

### Confidence intervals

Here, I want to show you how you WOULD calculate a CI by hand. In practice, I never do this. But it's useful to see how it could be done.

#### Get the data

Taking one high_n experiment for now

```
tea_dataset = tea_data_highn
```

```
tea_ratings <- filter(tea_dataset, condition == "tea first")$rating
milk_ratings <- filter(tea_dataset, condition == "milk first")$rating

# could also do in a pipe like so, but then you have to grab the column below, as in tea_rati
# tea_ratings <- tea_data_highn %>%
#   filter(condition=="tea first") %>%
#   select(rating)
```

### Calculate a CI on the effect (difference between conditions)

First we're going to calculate a CI by hand.

```
n_tea <- length(tea_ratings)
n_milk <- length(milk_ratings)
sd_tea <- sd(tea_ratings)
sd_milk <- sd(milk_ratings)

tea_sd_pooled <- sqrt(((n_tea - 1) * sd_tea ^ 2 + (n_milk - 1) * sd_milk ^ 2) /
                        (n_tea + n_milk - 2))

tea_se <- tea_sd_pooled * sqrt((1 / n_tea) + (1 / n_milk))


delta_hat <- mean(milk_ratings) - mean(tea_ratings)
tea_ci_lower <- delta_hat - tea_se * qnorm(0.975)
tea_ci_upper <- delta_hat + tea_se * qnorm(0.975)
```

To get the 95% CI with the t-distribution, you need to get the appropriate t-statistic from the distribution, which incorporates information about the degrees of freedom

The t-distribution is more appropriate when you have smaller sample sizes and is what is used in t.tests

```
num_observations = length(tea_dataset$rating)
df = num_observations-2 # for two sample t.test
tea_ci_lower_ttest <- delta_hat - tea_se * qt(0.975,df)
tea_ci_upper_ttest <- delta_hat + tea_se * qt(0.975,df)

c(tea_ci_lower_ttest, tea_ci_upper_ttest) # our by-hand 95% CI
```

```
[1] -0.02800509  1.19467176
```

The above code does this by hand - but it's to show you what this function is doing under the hood.

```
# Now the calculated CIs match those in the t-test outputs!
t.test(milk_ratings, tea_ratings, var.equal = TRUE)
```

```
    Two Sample t-test

data:  milk_ratings and tea_ratings
t = 1.9207, df = 46, p-value = 0.06098
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -0.02800509  1.19467176
sample estimates:
mean of x mean of y
 4.041667  3.458333
```

### Plotting CIs by hand for each condition

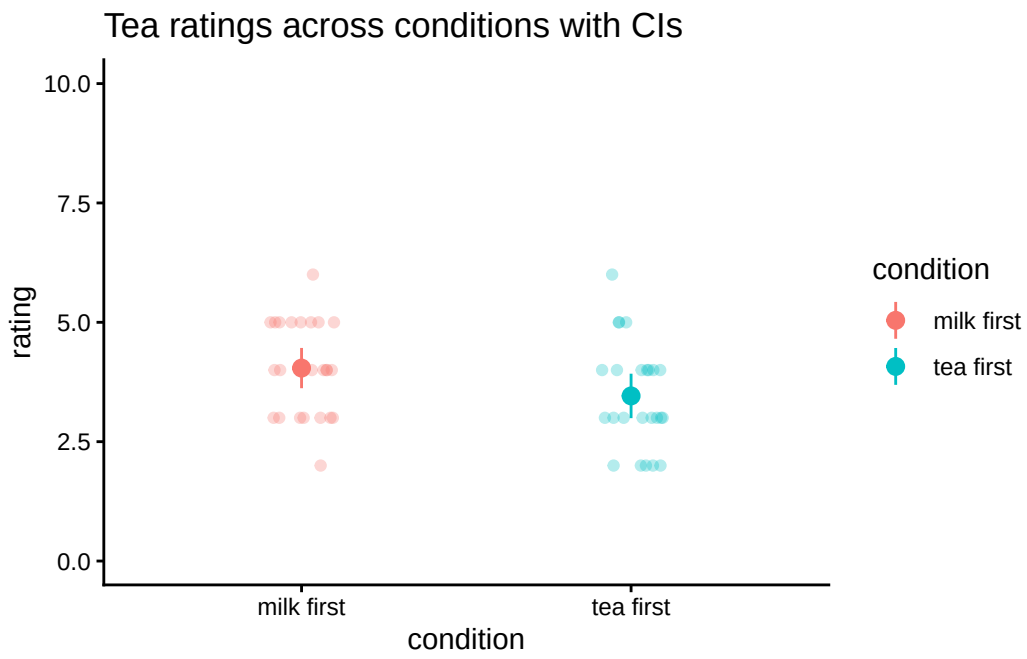
Also plot the SEs, and visualize how they're different

```
confidence_level=.95 # you can change this
# this formula below gives the critical t-value (as opposed to simply taken from the normal c
# qt(1 - (1 - confidence_level)/2, df = n - 1)

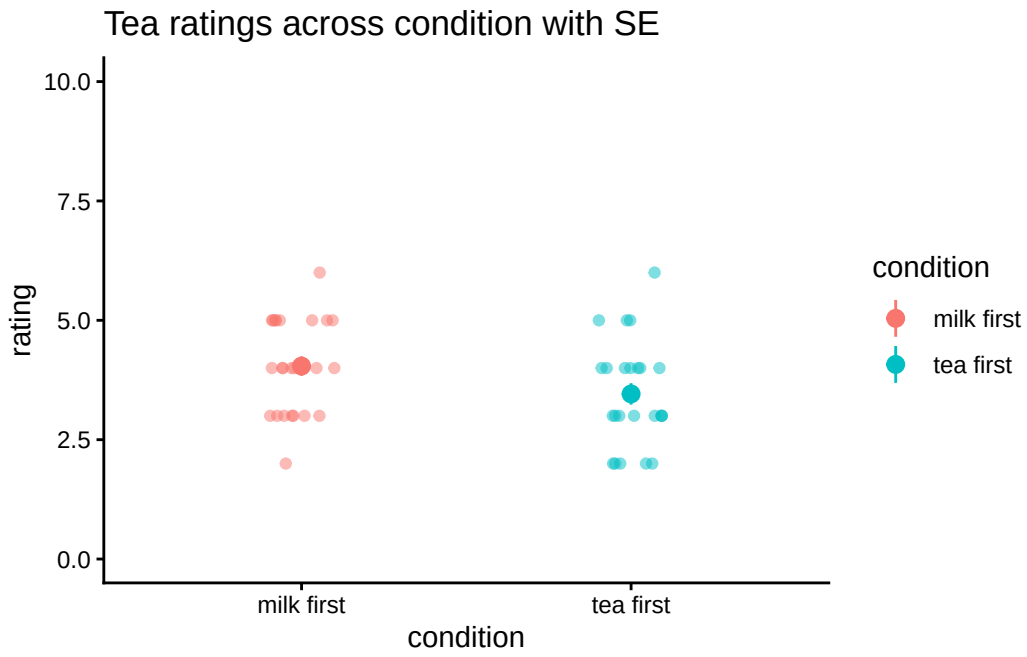
tea_data_highn_summary_cis <- tea_data_highn %>%
  group_by(condition) %>%
  summarize(cond_mean = mean(rating), cond_sd = sd(rating), n=length(rating)) %>%
  mutate(error = qt(1 - (1 - confidence_level)/2, df = n - 1)* (cond_sd/sqrt(n))) %>% # this
  mutate(ci_upper = cond_mean + error, ci_lower = cond_mean - error) %>%
  mutate(se_upper = cond_mean + cond_sd/sqrt(n), se_lower = cond_mean - cond_sd/sqrt(n))
```

Between subjects experiment – lots of variability! I like to visualize the raw data as well as the mean and CIs

```
ggplot(data = tea_data_highn, aes(x=condition, y=rating, col=condition))  +
  geom_jitter(width=.1, height=0, alpha=.3) + # visualizes all the raw data, with no variati
  theme_classic() +
  geom_pointrange(data = tea_data_highn_summary_cis, aes(x=condition, y = cond_mean, ymin = c
  ylim(0,10) +
  ggtitle('Tea ratings across conditions with CIs')
```




```
ggplot(data = tea_data_highn, aes(x=condition, y=rating, col=condition))  +
  geom_jitter(width=.1, height=0, alpha=.5) + # visualizes all the raw data, with no variatio
  theme_classic() +
  geom_pointrange(data = tea_data_highn_summary_cis, aes(x=condition, y = cond_mean, ymin = s
  ylim(0,10) +
  ggtitle('Tea ratings across condition with SE')
```




To do: What does each dot represent? What does the range represent in each graph? What does the confidence interval indicate? What does the SE indicate?

```
# each dot represents each simulation? The variability of means for each condition in 95% CI
```

To do: how does this change when you use the low-n experiment?

To do: Find the shortcut ggplot function so you don't have to calculate these by hand ever again :)

## Part 4: Simulating p-values across multiple experiments

To do: Comment what the lines of code are doing

#### When there is no actual effect

```
all_results=tibble()

for (this_sim in 1:1000) {
  this_experiment = null %>%
```

```
    filter(sim==this_sim)

  tea_ratings <- filter(this_experiment, condition == "tea first")$rating
  milk_ratings <- filter(this_experiment, condition == "milk first")$rating

  output = t.test(tea_ratings, milk_ratings, var.equal = TRUE)

  this_exp_output = tibble(pvalue = output$p.value)
  all_results = bind_rows(all_results, this_exp_output)

}
```

To do: Look at the distribution of p-values (hint: in all_results$pvalue)

```
all_results$pvalue
```

```
  [1] 0.0540825461 0.0663180478 0.1500687317 0.8543961045 0.6772119708
  [6] 0.4140321347 0.2651682381 0.7742415372 0.5360235897 0.5988885349
 [11] 0.2040528281 0.5195274393 1.0000000000 0.3305601237 0.3814304358
 [16] 0.3176832781 0.1651318674 0.4354283885 0.8279928566 0.0493415803
 [21] 0.2284933130 0.4896716611 0.0925856569 0.8743846669 0.5119031594
 [26] 0.6238815553 0.7602186279 0.7175282605 0.7718744457 0.4246893838
 [31] 0.2968767259 0.6607156572 0.2699113594 1.0000000000 0.3944191967
 [36] 0.0040962058 0.7302975251 0.7040767894 0.3787077381 0.7027564996
 [41] 0.7251907278 0.1649492762 0.8613655018 0.7700494553 0.0851155029
 [46] 0.2651682381 1.0000000000 0.2113289741 0.7566505701 0.7302975251
 [51] 0.1752011477 1.0000000000 1.0000000000 0.6986871217 0.3201639606
 [56] 1.0000000000 0.3464136279 0.8796547826 0.0214244810 1.0000000000
 [61] 0.2974849683 0.3255864404 0.3814304358 0.1147415092 0.4231605687
 [66] 0.3201639606 0.7718744457 1.0000000000 0.2157898096 0.4214263196
 [71] 0.4793480399 0.3524009381 0.6459810099 0.1237910356 0.7241341381
 [76] 0.0388602058 1.0000000000 0.8365817041 0.6587226780 0.2967071245
 [81] 0.6701827684 0.6878207459 0.3814304358 0.6238815553 0.8727470962
 [86] 0.7536785718 0.6414064827 0.6645992329 0.1370864492 1.0000000000
 [91] 0.4219632129 0.5851780839 0.6003297264 0.0557486053 0.1353595285
 [96] 1.0000000000 0.5268080717 0.7040767894 0.0484026929 0.0878069735
[101] 0.4644155320 0.1651318674 0.4946109250 0.5025368521 0.5286009603
[106] 0.5645120230 0.4828576969 0.1947310162 0.5215321238 0.2113865204
[111] 0.3952040906 0.7175282605 0.5588469767 0.8484195024 0.5144844114
[116] 0.6525265973 0.8562365447 0.7066654824 0.8338632496 0.0666805019
[121] 0.6986871217 0.3109237817 0.1183767043 0.0534335651 0.2589736835
```

```
[126] 0.4828576969 0.5902891157 0.8524831256 0.8290145436 0.8659819961
[131] 0.4420656952 0.8290145436 0.6342003430 0.3710545225 0.4246893838
[136] 0.3321949847 0.1820889790 0.7241341381 0.4285533081 0.2875914309
[141] 0.4879931516 0.5282839633 0.2101817437 1.0000000000 0.8727470962
[146] 1.0000000000 0.1136993218 0.4063541951 0.7388144342 0.7251907278
[151] 0.1680380540 0.8701660468 0.4626337944 0.6238815553 0.1370864492
[156] 0.0006206554 0.1466473154 1.0000000000 0.6297199688 0.6152248135
[161] 0.7175282605 0.3532052453 0.8178900275 0.3458614230 0.7536785718
[166] 0.2494135045 0.5086184970 0.2803543520 0.1021000109 0.0413103453
[171] 0.5666952195 0.8613655018 0.4606771166 0.6437157228 0.8365817041
[176] 0.0130011848 0.6003297264 0.8447638384 0.8003560437 0.8440004517
[181] 1.0000000000 0.8447638384 0.8597173170 0.0804795198 0.3900736887
[186] 0.8659819961 0.3045936199 1.0000000000 0.5069379372 0.2857877534
[191] 0.3464136279 0.1903642405 0.2755968360 0.6113278636 0.6900209827
[196] 0.7140423984 0.1867292196 0.6772119708 0.5086184970 0.3791309111
[201] 0.3100852141 0.1024947669 0.8290145436 0.4285533081 0.1344685306
[206] 0.3630224537 0.6944460839 1.0000000000 0.6183845101 0.2101817437
[211] 0.0598571401 0.2712245874 0.1845366658 0.4978260367 0.4088430727
[216] 0.7104243277 0.8739812071 0.6342003430 0.8562365447 0.5867709510
[221] 0.0474996513 0.0878069735 0.5988885349 0.4571459932 0.8836803497
[226] 0.8141051593 0.8543961045 0.8619021210 0.4344251908 0.0261994946
[231] 0.4775665478 0.0375448159 0.8916956029 1.0000000000 0.5360235897
[236] 0.4978260367 0.5549458296 0.0375448159 0.5867709510 0.4214263196
[241] 0.8664669588 0.5988885349 0.4010691218 1.0000000000 0.5115776265
[246] 0.8290145436 0.4721118074 0.3986997629 0.2101817437 0.3100137452
[251] 0.5337698917 0.4086597571 0.4946109250 0.6155443092 0.2572421777
[256] 0.3724217281 0.0127449757 0.3022444517 0.4484783124 0.4571459932
[261] 0.3787077381 1.0000000000 0.0745508850 0.3724217281 0.2713467764
[266] 0.2423806645 0.7465726899 0.8191011968 0.3709447733 0.6113278636
[271] 0.3649348334 0.0493415803 0.1984450786 0.7027564996 0.0813757766
[276] 1.0000000000 0.3176832781 0.0554584179 1.0000000000 0.1947310162
[281] 0.1478820708 0.7152186405 0.2498633855 0.3983745449 0.8178900275
[286] 0.3587560765 0.1984450786 0.6915172848 0.1500687317 0.5337698917
[291] 0.3009132542 0.0163210414 0.1572419343 0.0442960324 1.0000000000
[296] 1.0000000000 0.1986622720 0.5699653793 1.0000000000 0.3321949847
[301] 0.5119031594 0.8619021210 0.1136993218 0.5468246401 0.3767930540
[306] 0.5144844114 0.4285533081 1.0000000000 0.7140423984 0.8763454071
[311] 1.0000000000 1.0000000000 0.8524831256 0.5851780839 0.2713467764
[316] 0.0484605513 0.4775665478 0.1283644281 0.8659819961 0.3176832781
[321] 0.5752198374 0.6363805329 0.5215321238 0.3983745449 0.6986871217
[326] 0.6155443092 0.8644947791 0.0877922105 0.0024762894 0.8100740855
[331] 0.5433968778 0.0076707093 0.0851155029 0.1397261238 0.2069830904
[336] 0.3250497019 0.2712245874 0.4137305800 0.1649492762 0.1147415092
```

```
[341] 0.3751305196 0.7140423984 0.4879931516 0.0992679273 0.8365817041
[346] 0.6723790402 0.3464136279 0.0966319022 0.6915172848 0.1984450786
[351] 0.8484195024 0.3754667424 0.5337698917 0.8462575852 1.0000000000
[356] 0.2968767259 0.0892681442 0.8338632496 0.0272679755 0.1649492762
[361] 0.3814304358 0.3131211738 0.8543961045 0.2933865280 0.3201639606
[366] 0.2215515637 0.4321588686 1.0000000000 0.3814304358 0.8701660468
[371] 0.0770393713 1.0000000000 0.8613655018 0.6747257113 0.8778495682
[376] 0.5944699411 0.2651682381 0.5469549499 0.1018778930 0.8659819961
[381] 1.0000000000 0.3413951615 1.0000000000 0.2453321113 0.1551508463
[386] 1.0000000000 0.3009132542 0.7559176615 0.2682134281 0.8391709477
[391] 0.5469549499 1.0000000000 1.0000000000 0.4063541951 0.3109237817
[396] 0.0627719635 0.8374586292 0.0135621954 1.0000000000 0.3153167896
[401] 0.7643000439 0.5645120230 0.5549458296 0.2440709243 0.7241341381
[406] 0.4946109250 0.2728694428 0.3429402654 0.4277116605 0.7865554830
[411] 1.0000000000 0.6938024432 0.0944111696 0.8524831256 0.6126399324
[416] 0.0745021512 0.8504927144 1.0000000000 0.7423493720 0.3830578963
[421] 0.0048548860 0.3100852141 0.4866762675 0.3100852141 0.7272680142
[426] 0.3532052453 0.3205064871 0.2063342128 0.8391709477 0.0746495749
[431] 0.5360235897 0.7272680142 0.7465726899 0.2941493636 0.1097745670
[436] 0.8440004517 0.8864477429 0.7040767894 0.3209339594 0.1752011477
[441] 0.4285533081 0.4231605687 0.1736175094 0.4879931516 1.0000000000
[446] 0.4978260367 0.1784985479 0.8100740855 0.4626337944 0.1300095963
[451] 0.3787077381 0.3880313715 0.8511651094 0.1984450786 0.5504316335
[456] 0.8597173170 0.5322014288 0.0498646117 0.8739812071 0.5737071235
[461] 0.8440004517 0.0224959391 0.0646165332 0.4820217745 1.0000000000
[466] 0.7152186405 0.2968767259 1.0000000000 0.0851155029 0.7302975251
[471] 0.7681799679 0.5086184970 0.7498058561 0.6607156572 0.6317002415
[476] 0.0613786881 0.1395108736 0.1136993218 0.8191011968 0.1986622720
[481] 0.1362272761 0.8447638384 0.3787077381 0.8462575852 0.1370864492
[486] 1.0000000000 0.4793480399 0.0627719635 0.0002474543 0.4137305800
[491] 0.4238308910 1.0000000000 0.2990525473 0.0646165332 1.0000000000
[496] 0.8504927144 0.2294914040 0.0557486053 0.5851780839 0.3814304358
[501] 0.4773674880 0.1311736402 1.0000000000 0.2941493636 0.3608348779
[506] 0.6701827684 0.5802871392 0.0417305294 0.0952955088 0.5215321238
[511] 0.6525265973 0.3587560765 0.6330954745 0.7152186405 0.0981691776
[516] 0.5802871392 0.2990525473 0.4238308910 0.6900209827 0.8597173170
[521] 0.7152186405 0.3391290814 0.8524831256 0.2284933130 0.0663180478
[526] 0.8391709477 0.4063541951 0.2428549404 0.2968767259 0.1601721201
[531] 0.5902891157 0.0892681442 0.0081001069 0.6297199688 0.3814304358
[536] 0.2630332266 0.8365817041 0.3814304358 1.0000000000 0.0854146308
[541] 0.6342003430 0.1736175094 0.1843495144 0.3321949847 0.4010691218
[546] 0.4285533081 0.3900736887 0.2428549404 0.6701827684 0.1705119792
[551] 0.5851780839 0.2941493636 0.6754968161 0.8597173170 0.3983745449
```

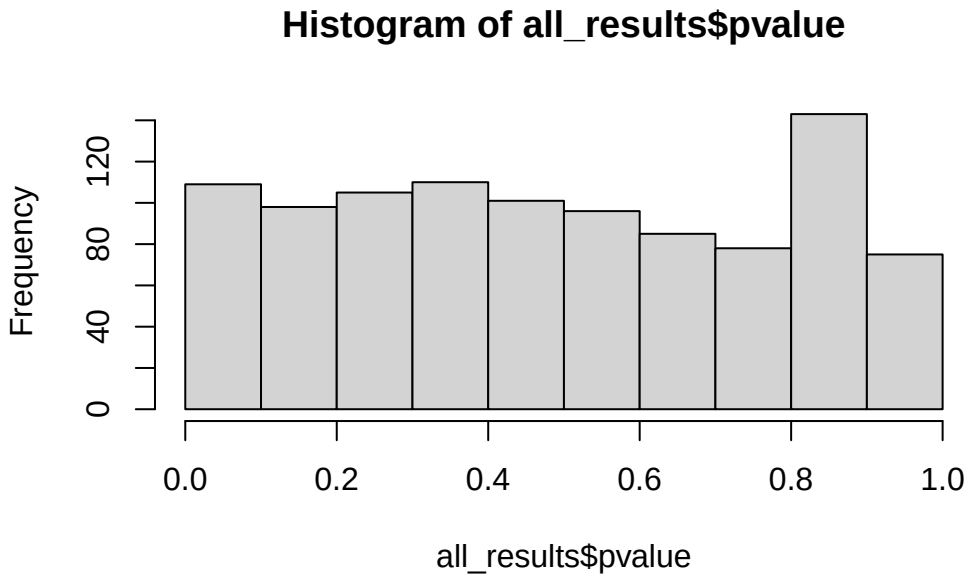
```
[556] 0.5717383647 0.6003297264 0.2453321113 0.8763454071 0.2440445904
[561] 0.2857877534 0.6003297264 0.3709447733 0.6805623484 0.4896716611
[566] 0.2044730632 0.5752198374 0.6900209827 0.8338632496 0.8644947791
[571] 0.0656892141 0.7272680142 0.6571481112 0.0569834242 0.0683203030
[576] 0.6772119708 0.2288880184 0.6003297264 0.3791309111 1.0000000000
[581] 0.0509007508 0.5635829113 0.8365817041 0.3608348779 0.8751800961
[586] 0.8072363244 0.4354283885 0.5571528186 0.5988885349 0.4219632129
[591] 0.1024947669 0.8214527706 0.0345996767 0.5538299631 0.4664850192
[596] 0.1370864492 0.2428549404 0.4449141903 1.0000000000 0.5268080717
[601] 0.1651318674 0.6805623484 1.0000000000 0.8279928566 0.8597173170
[606] 0.0484605513 0.7241341381 0.2215515637 0.1018778930 0.0484605513
[611] 0.6944460839 0.5978886568 0.4664850192 0.5970650102 1.0000000000
[616] 1.0000000000 1.0000000000 0.7414791008 0.5268080717 0.1278468683
[621] 0.4775665478 0.4140321347 0.1068263026 0.8846251722 0.5268080717
[626] 0.2857877534 0.8310044679 0.1397261238 0.8524831256 0.6363805329
[631] 0.8374586292 0.3458614230 0.5337698917 0.0121093388 0.6426949041
[636] 0.7819810255 0.3009132542 1.0000000000 0.0686424986 0.0743178893
[641] 0.1183767043 0.2474217823 0.2240168330 0.2847449528 0.2682134281
[646] 0.1311736402 0.8374586292 0.0701782542 0.8163412969 0.8629569276
[651] 0.6944460839 0.2267106861 0.8178900275 0.4866762675 0.1278468683
[656] 0.4946109250 0.7423493720 0.5144844114 0.4664850192 0.3791309111
[661] 0.1397261238 0.6031662724 0.4442275821 0.7251907278 0.1936797675
[666] 0.5717383647 0.1649492762 0.4978260367 0.8714760043 0.6238815553
[671] 0.5699653793 1.0000000000 1.0000000000 0.7251907278 1.0000000000
[676] 0.5195274393 0.3983745449 0.5282839633 0.3201639606 0.7850627973
[681] 0.3626664825 0.8763454071 0.2240168330 0.1483962415 0.1903642405
[686] 0.2516836779 0.1309328009 0.2267106861 0.3524009381 0.7423493720
[691] 0.7175282605 0.1905345663 0.4420656952 0.5215321238 0.3814304358
[696] 0.0627719635 0.2713467764 0.8391709477 0.0348165044 0.7908536384
[701] 0.8462575852 0.0579233740 1.0000000000 0.4664850192 0.0356525351
[706] 0.1748651896 0.5529562753 0.0222935866 0.7536785718 0.8290145436
[711] 0.5645120230 0.3402019470 0.8619021210 0.8562365447 0.5851780839
[716] 0.8279928566 0.4793480399 0.0347548825 0.8440004517 0.4879931516
[721] 0.1500687317 0.0683203030 0.6031662724 0.2589736835 0.6701827684
[726] 0.0817913074 0.4619318091 0.4038344341 0.1935860205 0.8374586292
[731] 0.2529370069 0.0569596555 0.5867709510 0.0981691776 0.7175282605
[736] 0.2395185205 0.8893023935 0.6226804720 0.6437157228 0.5571528186
[741] 0.5802871392 0.2284933130 0.2682134281 0.5645120230 0.4354283885
[746] 1.0000000000 0.3983745449 0.8706071396 0.2288880184 0.3630224537
[751] 0.8279928566 0.4238308910 0.5086184970 0.0250194761 0.0372632913
[756] 0.2474217823 0.4866762675 1.0000000000 0.4503185072 0.5119031594
[761] 0.0410273076 0.2630332266 0.8141051593 0.3131211738 0.6226804720
[766] 0.2755968360 0.6155443092 0.8290145436 0.8867431879 0.2712245874
```

```
[771] 0.3724217281 0.3787077381 0.8688152019 0.5337698917 1.0000000000
[776] 0.0878069735 0.3391290814 0.3724217281 0.5717383647 0.7440638823
[781] 0.2494135045 0.0256706038 0.8279928566 0.4644155320 0.3791309111
[786] 0.0456071490 0.0255334690 0.1160791720 0.2968767259 0.1365753851
[791] 0.2836542242 1.0000000000 0.8504927144 1.0000000000 0.6853981247
[796] 1.0000000000 0.1370864492 0.3170591022 0.8613655018 1.0000000000
[801] 0.7152186405 0.2288880184 0.5360235897 0.4140321347 0.6073104542
[806] 0.4442275821 0.8980397054 0.3983745449 0.8664669588 0.3539474199
[811] 0.0151509426 0.5988885349 0.0639518675 0.0036601664 0.5445921391
[816] 0.7027564996 0.5529562753 0.2968767259 0.8374586292 0.0387589090
[821] 0.8511651094 0.1183767043 0.8290145436 0.2040528281 0.6598342019
[826] 0.2861921550 0.2440709243 0.8462575852 0.5802871392 0.8688152019
[831] 0.6607156572 0.5717383647 0.0569596555 0.7465726899 0.0985405928
[836] 1.0000000000 0.2267106861 0.3880313715 0.1560853259 0.2240168330
[841] 0.3880313715 0.4786376045 0.2401470389 0.1021000109 0.2728694428
[846] 0.1379098499 0.7251907278 0.1572419343 0.5645120230 1.0000000000
[851] 0.2651682381 0.3201639606 0.3669006348 1.0000000000 0.0256706038
[856] 0.5071188144 0.2620252826 0.0335555569 0.3934338922 0.4828576969
[861] 0.8290145436 0.3824864493 0.8839978648 1.0000000000 0.8447638384
[866] 0.8778495682 1.0000000000 0.4214263196 0.5529562753 0.7332283340
[871] 0.6190071864 0.1184214587 0.6390518530 0.2857877534 0.8739812071
[876] 0.4137305800 0.2478898497 0.6701827684 0.5717383647 0.2990525473
[881] 0.6238815553 0.2588457956 0.8057689054 0.4606771166 0.8796547826
[886] 0.6515797083 0.2288880184 0.8613655018 0.2360489821 0.0482855698
[891] 0.6395791025 0.1416432682 0.7962004027 0.8613655018 0.8619021210
[896] 0.8739812071 0.0110482809 0.5215321238 0.5159186486 0.8839978648
[901] 0.7643000439 0.4866762675 0.4619318091 0.1986622720 0.7272680142
[906] 0.7104243277 0.2642214115 0.7629628073 0.6754968161 0.2941493636
[911] 0.7629628073 1.0000000000 0.4063541951 0.0551581640 0.8568346890
[916] 0.8290145436 0.1764631968 0.5445921391 1.0000000000 0.7302975251
[921] 0.6238815553 0.4866762675 0.1054138823 1.0000000000 0.1752011477
[926] 0.4793480399 0.0842180436 0.4503185072 0.4546783676 0.6915172848
[931] 0.3880313715 0.8613655018 0.3354308843 0.8462575852 0.0375448159
[936] 0.2215515637 0.2101817437 0.3494572820 0.1309328009 0.0084794994
[941] 0.1136993218 0.2010959340 0.5195274393 0.3448764166 0.7027564996
[946] 0.0877922105 0.8374586292 0.6598342019 0.4442275821 0.8674212974
[951] 0.0015398879 0.7440638823 0.8504927144 0.8524831256 0.8462575852
[956] 0.7688081857 0.1905345663 0.3321949847 0.8785813533 0.7908536384
[961] 0.1927480354 0.3413951615 0.4420656952 0.2836542242 0.0739033739
[966] 0.1227031218 0.4063541951 0.2699113594 0.6317002415 0.3201639606
[971] 0.1889985930 0.3814304358 0.2588457956 0.7066654824 0.3986997629
[976] 0.1466473154 0.3131211738 0.7140423984 0.1572419343 0.6853981247
[981] 0.5867709510 0.8214527706 0.6183845101 1.0000000000 0.4664850192
```

```
 [986] 0.0981691776 0.7251907278 0.0949705751 0.7302975251 0.8543961045
 [991] 0.4489819846 1.0000000000 0.8524831256 0.1208135159 0.8796547826
 [996] 0.4828576969 0.6155443092 0.2292743254 0.4664850192 0.1483962415
```

Make a histogram

```
hist(all_results$pvalue)
```

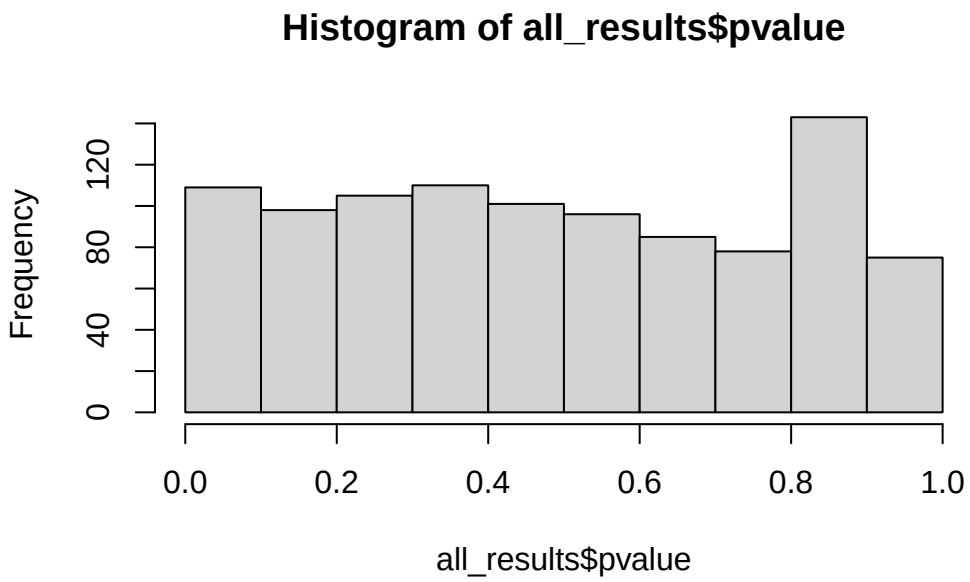



What is the distribution of p-values when the null is true?

```
# rectangular distribution?
```

Calculate the proportion of p-values that are less than .05 What was our false positive rate?

```
# you'll need all_results$pvalue
hist(all_results$pvalue)
```




```
mean(all_results$pvalue <.05)
```

```
[1] 0.051
```

**Now for an experiment when there is actually an effect**

```
all_results_high_n=tibble()

for (this_sim in 1:1000) {
  this_experiment = samps_highn %>%
    filter(sim==this_sim)

  tea_ratings <- filter(this_experiment, condition == "tea first")$rating
  milk_ratings <- filter(this_experiment, condition == "milk first")$rating

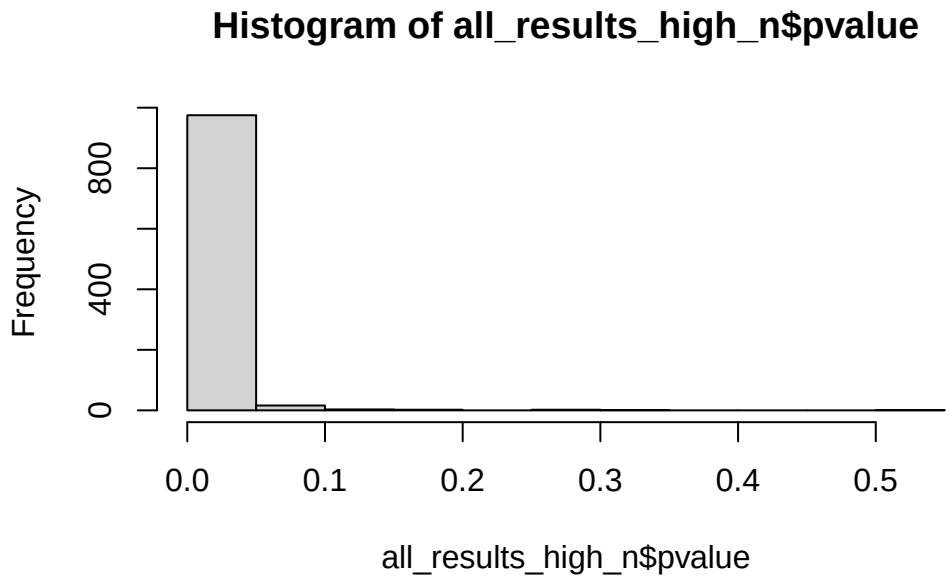
  output = t.test(tea_ratings, milk_ratings, paired = FALSE, var.equal = TRUE)

  this_exp_output = tibble(pvalue = output$p.value)
  all_results_high_n = bind_rows(all_results_high_n, this_exp_output)

}
```

How often did we fail to reject the null hypothesis? When was our p-value greater than p=.05? What does the distribution of p-values look like?

```
# you'll need all_results_high_n$pvalue
hist(all_results_high_n$pvalue)
```




```
mean(all_results_high_n >= .05)
```

```
[1] 0.025
```

## Part 5: Power curves

In Part 4 you counted how often experiments with 48 participants and a true effect of delta = 1.5 came out significant. That proportion is the study's **power**: the probability of getting p < .05 when there really is an effect. Here you will estimate power for many sample sizes and effect sizes at once, and plot it.

`run_one_experiment()` simulates one experiment with our `make_tea_data()` function and returns its p-value. Run this chunk a few times: you get a different p-value each time.

```
run_one_experiment <- function(n_total, delta) {
  this_experiment <- make_tea_data(n_total = n_total, delta = delta)
  t.test(rating ~ condition, data = this_experiment, var.equal = TRUE)$p.value
}

run_one_experiment(n_total = 48, delta = 1.5)
```

```
[1] 5.795479e-07
```

To do: Fill in `estimate_power()` so that it runs `n_sims` experiments and returns the proportion of them with p < .05. Hint: `map_dbl(1:n_sims, \(i) run_one_experiment(n_total, delta))` gives you a vector of `n_sims` p-values.

Check your function: `estimate_power(48, 1.5)` should be close to 1 minus the miss rate you found in Part 4, and `estimate_power(48, 0)` should be close to .05.

```
estimate_power <- function(n_total, delta, n_sims = 500) {p_values <- map_dbl(1:n_sims, \(i)
mean(p_values < .05)
}

estimate_power(n_total = 48, delta = 1.5)
```

```
[1] 0.986
```

```
estimate_power(n_total = 48, delta = 0)
```

```
[1] 0.032
```

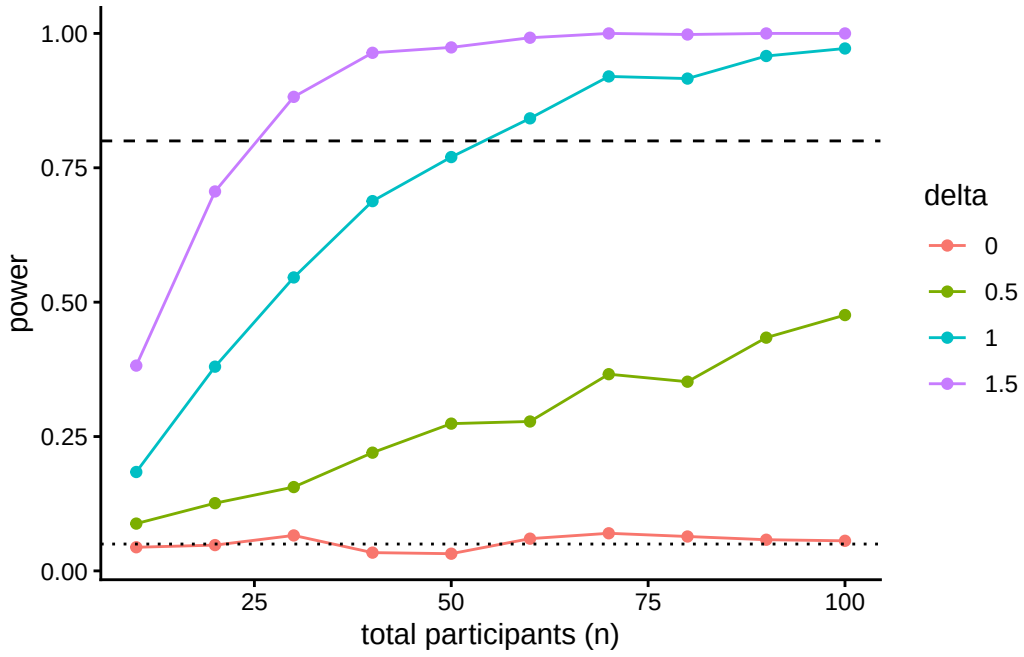
Now estimate power for every combination of sample size and effect size. `expand_grid()` makes a tibble with one row per combination, and `map2_dbl()` calls `estimate_power()` once per row. This runs 20,000 simulated experiments, so give it about 30 seconds.

```
power_curve <- expand_grid(n_total = seq(10, 100, by = 10), # total participants (keep these
                           delta = c(0, 0.5, 1, 1.5)) |>
  mutate(power = map2_dbl(n_total, delta, \(n, d) estimate_power(n, d)))
```

To do: Plot the power curves: `n_total` on the x-axis, `power` on the y-axis, and one line per `delta` (hint: `color = factor(delta)`). Add a dashed horizontal line at .8, the conventional target for power (hint: `geom_hline()`).

```r
power_plot <- ggplot(power_curve, aes(x = n_total, y = power, color = factor(delta))) +
  geom_line() +
  geom_point() +
  geom_hline(yintercept = .8, linetype = "dashed") +
  geom_hline(yintercept = .05, linetype = "dotted") +
  labs( x= "total participants (n)",
        y = "power",
        color = "delta") +
  theme_classic()

print(power_plot)
```




To do: Answer these questions.

```r
1.  What is "power" when delta = 0? Why doesn't it go up with more participants?
# it's zero. because there's no difference between these two groups, which is very unlikely t

2.  About how many participants do you need for 80% power when delta = 1? When delta = 0.5?
# For delta = 1, it should be over 75 participants. For delta = 0.5, I don't know about the e

3.  In Part 1, `tea_data` had 18 participants and a true effect of delta = 0.5. About what wa
# it must be less than 0.1? I wouldn't trust it that much.
```

```
Error in parse(text = input): <text>:1:5: unexpected symbol
1: 1.  What
        ^
```

**Bonus** Extend `n_total` until delta = 0.5 reaches 80% power. How many participants does it take?

## Exercises

1. Now go back (earlier in the code) and modify the DELTA in the simulation functions to be smaller so that there is only a small difference between groups. Is it still significant?
2. Rewrite this code with the smaller sample size simulations. What changes?